

Cahier de lab 4

Diffraction

Travail présenté à :
Daniel Côté & Michèle Desjardins

1 Objectifs

- Identifier les effets distincts de la diffraction et de l'interférence.
- Utiliser un patron de diffraction et d'interférence pour obtenir de l'information sur les objets causant ces phénomènes.
- Analyser les figures obtenues à l'aide de Fiji.
- Ouvrir un fichier de données dans Python et produire des figures de qualité.

2 Théorie

2.1 Définitions et Explications Sommaires

- **Diffraction** : Déviation d'une onde lumineuse lorsqu'elle rencontre une ouverture ou un obstacle de grandeur comparable à sa longueur d'onde. Le phénomène s'explique par le principe de Huygens qui considère chaque point de l'ouverture comme source secondaire.
- **Interférences** : Superposition d'ondes cohérentes qui conduit à des zones d'intensité renforcée (interférence constructive) ou annulée (interférence destructive).
- **Patron de Diffraction** : Patron caractérisé par un maximum central et des maxima secondaires, issus de la déviation de l'onde après le passage à travers ou autour d'un obstacle qui cause une diffraction.
- **Figure d'interférence** : Le patron caractérisé par des taches lumineuses (maxima) et des taches sombres (minima). Lorsque la lumière provient de plusieurs sources cohérentes (par exemple, deux fentes), les ondes se superposent et interfèrent constructivement ou destructivement.
- **Diffraction Circulaire** : Pour une ouverture circulaire, le patron (ou patron d'Airy) présente un disque central entouré d'anneaux décroissants en intensité.

2.2 Équation

Les équations pertinentes se trouvent à la figure 1. Dans ces expressions, l'angle θ est mesuré à partir du centre de l'ouverture, en prenant comme référence la direction de propagation de l'onde incidente. Les formules définissant les angles $\theta_{n,\min}$ et $\theta_{n,\max}$ s'appliquent lorsque leur valeur absolue ne dépasse pas $\pi/2$ rad.

Cas particulier	Figure observée	I / I_0	$\theta_{n \min}$	$\theta_{n \max}$
1) Une fente de largeur a (fig. 2)	de diffraction	$\left(\frac{\sin\left(\frac{\pi a \sin \theta}{\lambda}\right)}{\frac{\pi a \sin \theta}{\lambda}} \right)^2$	$\arcsin\left(\frac{m_n \lambda}{a}\right)$ $m_n = \dots -3, -2, -1, 1, 2, 3 \dots$ $n \neq 0$	$\arcsin\left(\frac{m_n \lambda}{a}\right)$ $m_n = \dots -3.471, -2.459, -1.430,$ $0, 1.430, 2.459, 3.471 \dots$
2) Une ouverture circulaire de rayon r	de diffraction	$\left(\frac{2J_1\left(\frac{2\pi r \sin \theta}{\lambda}\right)}{\frac{2\pi r \sin \theta}{\lambda}} \right)^2$	$\arcsin\left(\frac{m_n \lambda}{2r}\right)$ $m_n = \dots -3.238, -2.233, -1.220,$ $1.220, 2.233, 3.238 \dots$ $n \neq 0$	$\arcsin\left(\frac{m_n \lambda}{2r}\right)$ $m_n = \dots -3.699, -2.679, -1.635,$ $0, 1.635, 2.679, 3.699 \dots$
3) Deux fentes infiniment étroites séparées par une distance d (fig. 3)	d'interférence	$\cos^2\left(\frac{\pi d \sin \theta}{\lambda}\right)$	$\arcsin\left(\frac{m_n \lambda}{d}\right)$ $m_n = \dots -2.5, -1.5, -0.5,$ $0.5, 1.5, 2.5 \dots$ $n \neq 0$	$\arcsin\left(\frac{m_n \lambda}{d}\right)$ $m_n = \dots -3, -2, -1, 0, 1, 2, 3 \dots$
4) N fentes de largeur a disposées en réseau et séparées l'une de l'autre par une distance d	de diffraction et d'interférence	$\left(\frac{\sin\left(\frac{\pi a \sin \theta}{\lambda}\right)}{\frac{\pi a \sin \theta}{\lambda}} \right)^2 \left(\frac{\sin\left(\frac{N\pi d \sin \theta}{\lambda}\right)}{N \sin\left(\frac{\pi d \sin \theta}{\lambda}\right)} \right)^2$		

FIGURE 1 – Équations utilisées lors du laboratoire.

3 En lab

...

4 Code pour les fentes (ou antifentes) simple

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

def analyze_diffraction_with_threshold(
    x_pixels,
    intensity,
    pixel_to_m,
    wavelength,
    L,
    n_minima=3,
    intensity_threshold=0.7,
    distance_points=5,
    prominence_val=0.02,
    filename = "res"
):
    """
    Calcule la largeur de la fente à partir du patron de diffraction.

    x_pixels : positions en pixels.
    intensity : intensités mesurées.
    pixel_to_m : conversion pixels -> mètres.
    wavelength : longueur d'onde (m).
    L : distance fente-écran (m).
    n_minima : nombre de minima par côté.
    intensity_threshold : seuil d'intensité.
    distance_points : distance minimale entre minima.
    prominence_val : prominence minimale.

    Return:
    slit_width_estimate : largeur estimée (m) ou None.
    x_m_centered : positions centrées sur le maximum.
    intensity_norm : intensité normalisée.
    """

    # Trouver indice maximum central
    index_max = np.argmax(intensity)

    # Conversion en mètres
    x_m = x_pixels * pixel_to_m
```

```
# Normalisation intensité
intensity_norm = intensity / np.max(intensity)

# Récupérer le maximum central
x_max_m = x_m[index_max]

# Centrer sur le maximum central
x_m_centered = x_m - x_max_m

# Détection initiale des minima
minima_indices, properties = find_peaks(
    -intensity_norm,
    distance=distance_points,
    prominence=prominence_val
)

# Filtrage par intensité : garde juste les minima < intensity_threshold
minima_indices_filtered = [
    idx for idx in minima_indices
    if intensity_norm[idx] < intensity_threshold
]

minima_indices_filtered = np.array(minima_indices_filtered)

# Séparer minima gauche/droite par rapport au maximum
left_mask = x_m_centered[minima_indices_filtered] < 0
right_mask = x_m_centered[minima_indices_filtered] > 0

left_minima_indices = minima_indices_filtered[left_mask]
right_minima_indices = minima_indices_filtered[right_mask]

# Trier par distance au centre
left_minima_indices = left_minima_indices[np.argsort(np.abs
(x_m_centered[left_minima_indices]))]
right_minima_indices = right_minima_indices[np.argsort(np.abs
(x_m_centered[right_minima_indices]))]

# Considérer n minimas à gauche et à droite pour calculer
left_minima_indices = left_minima_indices[:n_minima]
right_minima_indices = right_minima_indices[:n_minima]

# Calcul de largeur
slit_widths = []
```

```
nb_pairs = min(len(left_minima_indices), len(right_minima_indices))
for i in range(nb_pairs):
    m = i + 1 # L'ordre du minimum
    x_left = np.abs(x_m_centered[left_minima_indices[i]])
    x_right = np.abs(x_m_centered[right_minima_indices[i]])
    y_avg = (x_left + x_right) / 2.0
    # Formule:  $a = m * \lambda * L / y_{avg}$ 
    a_estimate = m * wavelength * L / y_avg
    slit_widths.append(a_estimate)

# Moyenne de largeur
slit_width_estimate = np.mean(slit_widths)
print(f"Largeur calculé : {slit_width_estimate:.3e} m")

# Plot
plt.figure(figsize=(8, 5))
plt.scatter(x_m_centered, intensity_norm, s=1, label='Intensité normalisée')
plt.xlabel("Position [m]")
plt.ylabel("Intensité relative [-]")

# Minima (après filtrage)
plt.plot(x_m_centered[minima_indices_filtered],
         intensity_norm[minima_indices_filtered],
         'rs', markersize=4, label='Minima identifié')

# Annoter la largeur estimée
if slit_width_estimate is None:
    plt.text(0.05, 0.95,
            f"Largeur fente {slit_width_estimate:.3e} m",
            transform=plt.gca().transAxes,
            fontsize=12, verticalalignment='top',
            bbox=dict(facecolor='white', alpha=0.8, edgecolor='gray'))

plt.grid(False)
plt.margins(0)
plt.legend()
plt.savefig(filename, dpi=600, bbox_inches='tight')
plt.show()

return slit_width_estimate, x_m_centered, intensity_norm
```

5 Code pour doubles fentes

```
import numpy as np
import matplotlib.ticker as tck
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from getcsvData import getcsvData_dict

def determine_maxima(scheme_dict, measure, screen_slit, longueur_onde):
    Distance_pixels, Gray_Value = scheme_dict.values()

    pos = np.array(Distance_pixels)
    pos_fixed = np.arange(-measure/2, measure/2, measure/len(pos))
    gray_scale = np.array(Gray_Value)

    # reperer les piques du schema
    peaks = find_peaks(gray_scale, threshold=0)[0]
    peaks_pos = np.zeros(len(peaks))
    peaks_int = np.zeros(len(peaks))

    for i, j in enumerate(peaks):
        peaks_pos[i] = pos_fixed[j]
        peaks_int[i] = gray_scale[j]

    # identifier visuellement le premier pique a partir du centre
    ## fig_ident, ax_ident = plt.subplots()
    ## ax_ident.scatter(pos_fixed, gray_scale, s=1)
    ## ax_ident.scatter(peaks_pos, peaks_int)
    ## plt.show()

    index = int(input('index du premier maximum: '))
    pos_peak_1 = peaks_pos[index]
    print(f'distance du premier pique: {pos_peak_1:.3e} [m]')

    d = np.sqrt(pow(1/(pos_peak_1/screen_slit), 2) + 1)*longueur_onde
    print(f'distance d = {d:.3e} [m]')

    fig_display, ax_display = plt.subplots()

    ax_display.axvline(x=peaks_pos[index], color='red', linestyle=':', linewidth=1)
    ax_display.scatter(pos_fixed, gray_scale/gray_scale.max(), s=1)
    ## xmin, xmax = ax_display.get_xlim()
```

TPOP - Diffraction

```
##      ax_display.hlines(y=peaks_int[index]/gray_scale.max(), xmin=xmin, xmax=xmax, color
      ax_display.scatter(peaks_pos[index], peaks_int[index]/gray_scale.max(), color='green'

      # formatage du graphique
      ax_display.xaxis.set_minor_locator(tck.AutoMinorLocator())
      ax_display.yaxis.set_minor_locator(tck.AutoMinorLocator())
      ax_display.set_xlabel(r"Position sur l'écran [m]")
      ax_display.set_ylabel(r"Intensité relative [-]")
##      ax_display.margins(0)
      plt.show()

##      fig_display.savefig("figures_interference/R_50_10.pdf", format="pdf")

      return peaks_pos, peaks_int

if __name__ == "__main__":
    '''
    ---longueurs d'ondes---
    laser vert: 535 [nm]
    laser rouge: 650 [nm]
    '''
    vert = 535e-9
    rouge = 650e-9

    fich_v_100_15 = '_two_SLIT/v_double_1m_15cm.csv'
    fich_v_75_15 = '_two_SLIT/V_0.75m_15cm.csv'
    fich_v_50_15 = '_two_SLIT/V_0.5_15cm.csv'
    fich_r_50_10 = '_two_SLIT/R_0.5_10cm.csv'

    data_V_100 = getcsvData_dict(fich_v_100_15) # indice=5, threshold=1
    data_V_75 = getcsvData_dict(fich_v_75_15) # indice=29, threshold=0
    data_V_50 = getcsvData_dict(fich_v_50_15) # indice=38, threshold=0
    data_R_50_10 = getcsvData_dict(fich_r_50_10) # indice=25, threshold=0

    peak_pos, peak_int = determine_maxima(data_R_50_10, .1, .5, longueur_onde=rouge)

##      plt.show()
```